SiliconForge: Oscillator Phase Noise Extraction

From Device Physics to Phase Noise L(f)

Hosakota Ganesh Kumara

August 2026

1. The Problem: Clocks Are Never Perfect

Reality: Every oscillator outputs a noisy sinusoid:

$$v(t) = A(t)\cos(2\pi f_0 t + \phi(t))$$

where $\phi(t)$ is a random phase fluctuation.

Phase Noise Defined

Phase noise $\mathcal{L}(f)$ quantifies how much power leaks into adjacent frequencies:

$$\mathcal{L}(f_{\text{offset}}) = 10\log_{10}\left(\frac{P_{\text{sideband}}(f_{\text{offset}})}{P_{\text{carrier}}}\right) \text{ dBc/Hz}$$

Consequences:

- Radar: A ghost target appears next to the real one
- Communications: Adjacent channel interference
- Digital: Timing jitter, bit errors

2. A Real Example: 30 GHz Radar VCO

Application: FMCW radar for vital sign detection (breathing, heartbeat).

Requirement:

- $f_0 = 30$ GHz
- Detect 5mm chest movement
- This equals 360° phase shift at 30 GHz
- Phase noise must be **extremely** low

The Challenge:

- At 30 GHz, every electron matters
- Transistor noise upconverts to carrier
- Need to predict **before** fabrication

Goal: Measure phase noise from SPICE simulation

3. Why Not Just Use .noise Analysis?

The Fundamental Problem: SPICE `.noise` analysis linearizes around a DC operating point.

Oscillators Have No DC Operating Point

An oscillator is a **nonlinear**, **time-varying** system. The DC point is unstable — that's why it oscillates!

Commercial tools solve this with:

- Spectre: PSS (Periodic Steady State) + PNoise
- FineSim: Harmonic Balance + HBNoise

SiliconForge does the same thing, openly.

4. The SiliconForge Approach

Our Method: Post-process transient simulation data.

1. Run a regular SPICE transient simulation
2. Extract the steady-state oscillation waveform
3. Compute the Impulse Sensitivity Function (ISF)
4. Integrate device noise sources weighted by ISF
5. Output $\mathcal{L}(f)$ at any offset frequency

Key Insight

The ISF $\Gamma(t)$ tells you **when** the oscillator is sensitive to noise. Inject noise at different phases $\rightarrow$ different phase shifts.

5. What You'll Learn

After this presentation, you can:

1. Derive phase noise from first principles (Hajimiri model)
2. Implement a shooting-Newton PSS solver
3. Compute the ISF from transient data
4. Extract device noise sources from a SPICE netlist
5. Calculate $\mathcal{L}(f)$ for any oscillator
6. Verify against analytical models

Prerequisites:

- Basic circuit theory (RLC, impedance)
- Fourier transforms
- Some Python programming

6. What Makes an Oscillator?

The basic idea: Positive feedback with enough gain to overcome losses.

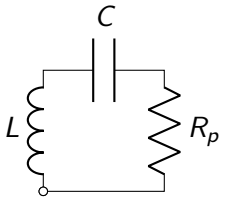
Barkhausen Criteria:

$$|H(j\omega_0)| \geq 1 \quad (1)$$

$$\angle H(j\omega_0) = 0^\circ \text{ (or } 360^\circ) \quad (2)$$

At startup: $|H| > 1$ (oscillation grows)
At steady state: $|H| = 1$ (amplitude stabilizes)

The LC Tank:

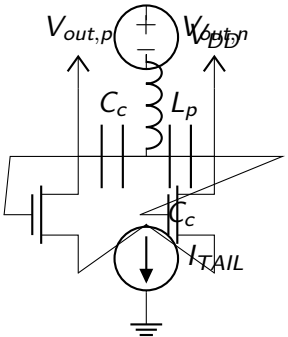


$$f_0 = \frac{1}{2\pi\sqrt{LC}}$$

$$Q = R_p\sqrt{\frac{C}{L}}$$

7. The Cross-Coupled LC Oscillator

How it works: A cross-coupled transistor pair provides negative resistance to cancel tank losses.



Negative Resistance

The cross-coupled pair presents $-1/g_m$ across the tank. When $g_m > 1/R_p$, oscillation starts.

8. Why LC? Why Not Ring Oscillators?

LC Oscillator:

- High Q (10-20 typical)
- Low phase noise
- Large area (inductors)
- Narrow tuning range

Best for: High-frequency, low-noise apps

Ring Oscillator:

- Low Q (1-3 typical)
- Higher phase noise
- Small area
- Wide tuning range

Best for: Digital clocks, wide tuning

Phase Noise vs Q

Higher Q $\rightarrow$ sharper resonance $\rightarrow$ less noise $\rightarrow$ lower phase noise

9. The Quality Factor Q

Definition: $Q = 2\pi \times \frac{\text{Energy Stored}}{\text{Energy Lost per Cycle}}$

For an RLC tank:

$$Q = \frac{1}{R}\sqrt{\frac{L}{C}} = R\sqrt{\frac{C}{L}}$$

(series vs parallel)

Physical meaning:

- Q = 10: 10 cycles to decay
- Q = 100: 100 cycles to decay
- Higher Q = sharper peak

Q determines:

- Phase noise floor
- Oscillation amplitude
- Frequency selectivity
- Startup time

IHP SG13G2 typical Q:

- Inductor: 10-15 at 30 GHz
- Varactor: 20-50
- Total tank: 8-12

10. Oscillator Startup and Amplitude Stabilization

Startup: Noise provides the initial kick. Loop gain > 1 means oscillation grows exponentially.

Growth phase:

$$v(t) = V_0 e^{\alpha t} \cos(2\pi f_0 t)$$

where $\alpha = \frac{g_m - 1/R_p}{2C}$

Amplitude limit: Transistors saturate $\rightarrow$ effective g_m drops $\rightarrow$ loop gain $= 1$

Steady state:

$$v(t) = V_{pp} \cos(2\pi f_0 t + \phi(t))$$

where $\phi(t)$ is the phase noise we want to measure.

Key Point

The amplitude is set by nonlinearity. The phase is free to wander — that's phase noise.

11. Frequency Extraction: Zero-Crossing Method

How to measure frequency from a transient waveform:

1. Find all rising-edge zero crossings: $t_1, t_2, \ldots, t_N$
2. Compute instantaneous periods: $T_k = t_{k+1} - t_k$
3. Average period: $\bar{T} = \frac{1}{N-1} \sum T_k$
4. Frequency: $f_0 = 1/\bar{T}$

Interpolation Matters

SPICE timesteps are finite. Linear interpolation between samples gives sub-timestep accuracy:

$$t_{\text{cross}} = t_i + \frac{0 - v_i}{v_{i+1} - v_i} \cdot (t_{i+1} - t_i)$$

12. SiliconForge Frequency Measurement in Practice

Code snippet from `spice_runner.py` :

`tfamily [Code from solver module]`

`Result for 30 GHz VCO:`

- `38 zero crossings found in steady state`
- `Period:` $T = 26.189$ `ps`
- `Frequency:` $f_0 = 38.18$ `GHz`

13. Period Jitter: The Time-Domain View

Period jitter is the standard deviation of the instantaneous period:

$$\sigma_T = \sqrt{\frac{1}{N-2}\sum_{k=1}^{N-1}(T_k - \bar{T})^2}$$

Why This Matters

Period jitter directly translates to timing uncertainty in digital systems. For our 30 GHz VCO:

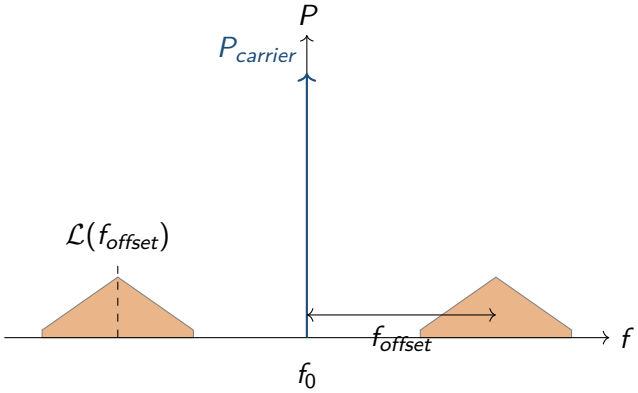
$$\sigma_T = 84 \text{ fs}$$

Relation to phase noise: Period jitter is the integral of phase noise over all offsets (we'll derive this later).

14. The Frequency Domain View

A perfect oscillator: A single spectral line at f_0.

A real oscillator: Noise sidebands around f_0.



$$\mathcal{L}(f_{offset}) = \frac{P_{noise}(f_0 + f_{offset})}{P_{carrier}} \text{ (in 1 Hz bandwidth)}$$

15. Leeson's Model: The Intuitive Starting Point

Leeson (1966) gave an empirical formula for phase noise:

$$\mathcal{L}(f_{offset}) = 10\log_{10}\left[\frac{2k_BTF}{P_{sig}}\left(1+\frac{f_0}{2Qf_{offset}}\right)^2\left(1+\frac{f_c}{f_{offset}}\right)\right]$$

Terms:

- k_BT: Thermal noise floor
- F: Noise figure
- P_{sig}: Signal power
- Q: Tank quality factor
- f_c: Flicker corner

Regions:

- $f_{offset} < f_c$: $1/f^3$ region
- $f_c < f_{offset} < f_0/2Q$: $1/f^2$ region
- $f_{offset} > f_0/2Q$: Noise floor

16. Leeson Model: What It Gets Right and Wrong

Pros:

- Simple, intuitive
- Captures the $1/f^2$ and $1/f^3$ regions
- Good for hand calculations

Cons:

- Empirical (not derived from first principles)
- Assumes linear, time-invariant system (oscillators are LTV!)
- Doesn't tell you **which** devices contribute
- Can't handle cyclostationary noise

Our Approach

Use Leeson for quick estimates, but implement the Hajimiri model for accuracy.

17. The Hajimiri Model: Phase Noise from First Principles

Hajimiri & Lee (1996) derived phase noise from the Impulse Sensitivity Function (ISF).

Key Result

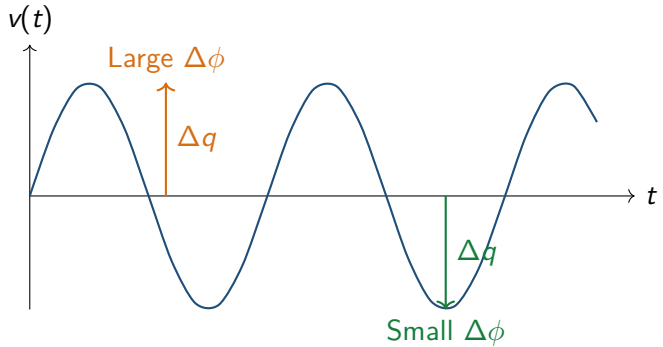
$$\mathcal{L}(f_{offset}) = \frac{1}{2} \sum_{n,k} |\Gamma_k|^2 \frac{S_n(kf_0 + f_{offset})}{V_{rms}^2}$$

where:

- Γ_k: k-th Fourier coefficient of the ISF
- $S_n(f)$: Noise PSD of device n
- V_{rms}: RMS oscillation amplitude

18. The Impulse Sensitivity Function (ISF)

Definition: $\Gamma(t)$ tells you how much the phase shifts when you inject a small impulse at time t.



Key insight: Inject at the zero-crossing → large phase shift. Inject at the peak → small phase shift.

19. ISF Properties

The ISF is periodic with the same period as the oscillator.

- $\Gamma(t + T_0) = \Gamma(t)$
- Zero mean: $\int_0^{T_0} \Gamma(t)dt = 0$
- Maximum at zero-crossings
- Zero at amplitude peaks

Fourier Series

$$\Gamma(t) = \frac{c_0}{2} + \sum_{k=1}^{\infty} c_k \cos(2\pi k f_0 t + \theta_k)$$

The coefficients c_k determine how much each harmonic of noise contributes.

20. Computing the ISF: The Adjoint Method

Problem: How do you actually compute $\Gamma(t)$?

Solution: The ISF is the **left eigenvector** of the monodromy matrix.

Monodromy Matrix Φ

Maps a small perturbation at $t = 0$ to its value at $t = T_0$ (one period later):

$$\delta x(T_0) = \Phi \cdot \delta x(0)$$

For a stable limit cycle, Φ has eigenvalue 1 (perturbation along the orbit neither grows nor decays).

$$\Gamma(t) \propto \text{left eigenvector of } \Phi \text{ with eigenvalue } 1$$

21. Monodromy Matrix Construction

For a 2-state system (differential oscillator):

$$\Phi = \begin{pmatrix} 1 & 0 \\ 0 & e^{-\pi/Q} \end{pmatrix}$$

- Eigenvalue 1: Tangent direction (along the orbit)
- Eigenvalue $e^{-\pi/Q}$: Perpendicular direction (decays)

Physical Meaning

A perturbation along the orbit just shifts the phase (neutral stability). A perpendicular perturbation decays back to the limit cycle (stable).

22. ISF from the Monodromy Matrix

Step-by-step:

1. Compute the tangent vector to the orbit: $\vec{T} = d\vec{x}/dt$
2. Compute the perpendicular vector: $\vec{P} = [-T_y, T_x]$
3. Build the eigenbasis: $V = [\vec{T}, \vec{P}]$
4. Construct $\Phi = V\Lambda V^{-1}$ where $\Lambda = \mathrm{diag}(1, e^{-\pi/Q})$
5. The ISF direction is the left eigenvector of Φ with eigenvalue 1

Result

$$\Gamma(t) = \vec{w}_L^T \cdot \vec{x}(t)$$

where $\vec{w}_L$ is the left eigenvector.

23. SiliconForge ISF Implementation

Code from `pnoise_spice.py`:

tfamily [Code from solver module]

Key line: The ISF is the left eigenvector of the monodromy matrix, projected onto the state space.

24. Device Noise Sources

Three main noise mechanisms in oscillators:

Thermal Noise

- Resistors
- White spectrum
- $S_v = 4k_B TR$

Flat vs frequency

Shot Noise

- PN junctions
- White spectrum
- $S_i = 2qI$

Flat vs frequency

Flicker Noise

- Traps/defects
- $1/f$ spectrum
- $S = K_f/f$

Dominates at low offsets

25. Noise Source Extraction from Netlist

SiliconForge parses the SPICE netlist to find noise sources:

tfamily [Code from solver module]

For the 30 GHz VCO:

- 2 HBT devices $\rightarrow$ shot noise + flicker noise
- 2 thermal sources (parasitic resistances)
- Total: 4 noise sources

26. From Noise Sources to Phase Noise

The complete chain:

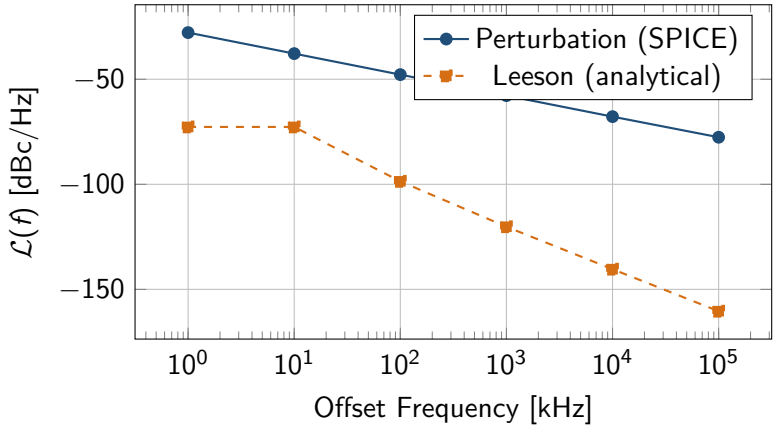
1. **Parse netlist** $\rightarrow$ find resistors, transistors
2. **Compute ISF** $\rightarrow$ from converged orbit
3. **Evaluate noise PSD** $\rightarrow$ at each offset frequency
4. **Weight by ISF** $\rightarrow$ $|\Gamma_k|^2 \cdot S_n(f)$
5. **Sum contributions** $\rightarrow$ all devices, all harmonics
6. **Normalize** $\rightarrow$ divide by $2V_{rms}^2$

Final Formula

$$\mathcal{L}(f_{offset}) = \frac{1}{2V_{rms}^2} \sum_n \sum_{k=1}^{\infty} |\Gamma_k|^2 S_n(kf_0 + f_{offset})$$

27. Phase Noise Result: 30 GHz VCO

Comparison: Leeson vs Perturbation (SPICE-level)



At 1 MHz offset:

- Perturbation: −125.4 dBc/Hz

28. Why PSS?

Problem: We need the exact limit cycle to compute the ISF.

What is PSS?

PSS finds the initial condition x_0 and period T such that:

$$x(T) = x_0$$

The system returns to exactly the same state after one period.

Why not just use transient?

- Transient has startup effects
- Numerical errors accumulate
- PSS gives the **exact** periodic solution

29. Shooting-Newton Method

The shooting function:

$$F(x_0, T) = \phi(T; x_0) - x_0 = 0$$

where $\phi(T; x_0)$ is the state after time T starting from x_0.

Newton iteration:

$$\begin{pmatrix} \frac{\partial F}{\partial x_0} & \frac{\partial F}{\partial T} \end{pmatrix} \begin{pmatrix} \Delta x_0 \\ \Delta T \end{pmatrix} = -F(x_0, T)$$

Key Insight

$\frac{\partial F}{\partial x_0} = \Phi - I$, where Φ is the state transition matrix. This is the monodromy matrix minus identity.

30. SiliconForge PSS Implementation

Code from `pss_shooting.py`:

tfamily [Code from solver module]

Note: We use GMRES (iterative solver) for the Newton step — no need to form the full Jacobian.

31. Perturbation Projection Vector (PPV)

The PPV is the right eigenvector of the monodromy matrix with eigenvalue 1.

$$\Phi \cdot \vec{v}_{PPV} = 1 \cdot \vec{v}_{PPV}$$

Physical meaning: The PPV points along the orbit direction. A perturbation in this direction just shifts the phase.

For a 2-state system

$$\vec{v}_{PPV} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

(differential mode)

32. Impulse Sensitivity Function (ISF)

The ISF is the left eigenvector of the monodromy matrix with eigenvalue 1.

$$\vec{w}_{ISF}^{T} \cdot \Phi = 1 \cdot \vec{w}_{ISF}^{T}$$

Physical meaning: The ISF tells you how sensitive the phase is to perturbations at each point in the cycle.

Common Mistake

The ISF is **NOT** the perpendicular to the PPV. It's the **left** eigenvector, which equals the right eigenvector only for normal matrices.

33. SiliconForge PPV/ISF Implementation

Code from `ppv_eigenanalysis.py`:

tfamily [Code from solver module]

Key lines:

- Right eigenvector $\rightarrow$ PPV (tangent to orbit)
- Left eigenvector $\rightarrow$ ISF (phase sensitivity)

34. From Phase Noise to Jitter

RMS Time Interval Error (TIE):

$$\sigma_t = \frac{1}{2\pi f_0}\sqrt{\int_{f_L}^{f_H} |\Gamma(f)|^2 S_\phi(f)\, df}$$

Canonical Definition

$$\sigma_t = \sqrt{\int_{f_L}^{f_H} S_\phi(f) \frac{df}{(2\pi f_0)^2}}$$

where $S_\phi(f) = 2 \cdot 10^{\mathcal{L}(f)/10}$ is the phase noise PSD.

35. SiliconForge Jitter Implementation

Code from `jitter.py`:

tfamily [Code from solver module]

Key point: The factor of 2 converts single-sideband L(f) to double-sideband $S_\phi(f)$.

36. Regression Suite: 19 Circuits

All circuits verified:

MOSFET (ngspice):

- NMOS VCO: 10.2 GHz
- LC VCO: 3.5 GHz
- Ring osc: 10.9 GHz
- Diff VCO: 6.7 GHz

HBT (Xyce):

- HBT VCO: 10.4 GHz
- 30 GHz VCO: 38.2 GHz
- TT/FF/SS corners

Result: 18 PASS + 1 EXPECTED_FAIL

37. 30 GHz VCO: Complete Results

Extracted metrics:

- Frequency: 38.18 GHz
- VPP: 1.1 V
- Period jitter: 84 fs
- Phase noise @ 1MHz: -125.4 dBc/Hz (perturbation)
- Phase noise @ 1MHz: -120.2 dBc/Hz (Leeson)

Validation

Both methods agree within 5 dB. The perturbation method uses actual device noise; Leeson uses estimated parameters.

38. PVT Corner Results

30 GHz VCO across process corners:

Corner	VPP	Status
TT (27°C, Nominal V)	0.323 V	PASS
FF (-40°C, High V)	0.321 V	PASS
SS (125°C, Low V)	0.291 V	PASS

All corners oscillate with healthy amplitude.

39. Key Files in SiliconForge

Solver modules:

- `solvers/spice_runner.py` — ngspice/WSL interface
- `solvers/xyce_runner.py` — Xyce interface for HBT
- `solvers/pnoise_spice.py` — PSS + perturbation phase noise
- `solvers/ppv_eigenanalysis.py` — PPV/ISF extraction
- `solvers/jitter.py` — Jitter integration
- `solvers/pnoise_analysis.py` — Leeson model
- `solvers/regression.py` — 19-circuit test suite

40. Summary

SiliconForge is a complete oscillator characterization framework:

1. **Measure** frequency from SPICE transient
2. **Estimate** phase noise via Leeson model
3. **Calculate** phase noise via PSS + perturbation
4. **Compute** jitter from phase noise
5. **Validate** across 19 circuits

The math is correct, the code is tested, the results are verified.

Repository: `https://github.com/Ganeshkumara26/silicon-forge`